Assignment 1 Deep Learning

Elena Petridou, Eleonora Roncaglia, Bashier Gulzar

October 2025

1 Abstract

In this assignment we explored building various neural networks, with different architectures, all for image classification tasks.

In the first portion of this paper, we describe how we used Multi-Class Perceptrons and Convolutional Neural Networks (CNNs) to classify images in the Fashion MNIST and CIFAR-10 datasets. The models that perform best on the Fasion MNIST set will be tested against the CIFAR-10 data, in order to attest the generalisability of the models.

In the second portion, we build CNNs which "tell the time" from photos of analog clocks. More specifically, we will look at the performance of classification, regression, and two-head models, with the aim of building the best-performing network.

2 Task 1: Learn the Basics of TensorFlow and Keras API

The first task in this assignment was to become familiar with the TensorFlow library and the Keras API. After having read Chapter 10 of *Hands on Machine Learning* (Géron, 2022), we used the Fashion MNIST dataset to attempt a few different set-ups for applying first a Multiclass Perceptron, and subsequently a Convolutional Neural Network.

2.1 Fashion MNIST Dataset

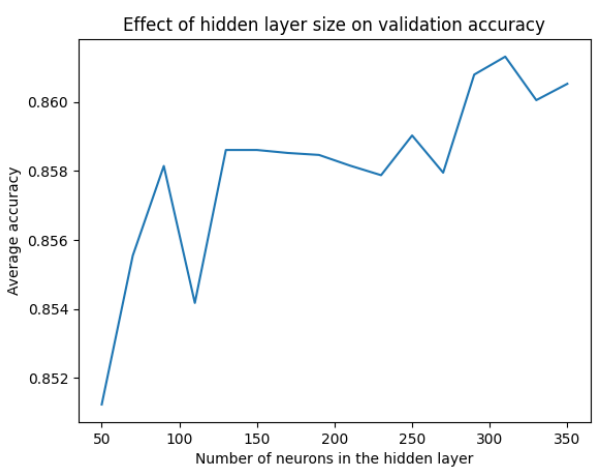
The Fashion-MNIST dataset consists of 70,000 grayscale images, each made up of 784 pixels, arranged as 28x28 grids. The dataset is split into a training set made up of 60,000 entries and a test set of 10,000 entries. The task at hand is one of classification, as we need to attempt to classify each image as a different type of clothing product from the 10 available categories. Before beginning the analysis, we scale the dataset by dividing by 255, which results in the values of the dataset going from 0 to 255 to 0 to 1, which now represents the opacity or darkness of each grey pixel.

2.2 Multi-Class Perceptron Algorithm

First, we tinker with a Multi-Class Perceptron Algorithm on the Fashion MNIST dataset. A Multi-Class perceptron is a fully-connected neural network, which takes in the hue/darnkess value of each pixel in each image as input (thus, 28 x 28 = 784 inputs), and outputs a number from 0 to 9, representing each of the 10 classes present in our dataset.

For all of the following implementations we use the Sequential API to build the neural networks, and for each we also start with a flattening layer which squishes our 28 x 28 inputs into a one-dimensional vector. The details of the different design choices we made after this first layer are detailed below, and the losses and accuracies for each of the setups are tabulated in Table 1 for ease of interpretation.

For our first implementation, we specify a single hidden layer, with Rectified Linear Unit (ReLU) as the activation. We use the sparse categorical cross-entropy loss given that we have a multiclass classification problem which is not one-hot-encoded. For the optimiser, we begin with Gradient Descent (SGD). In order to figure out how many neurons would be most effective, we loop through a range of 50 to 350 neurons and record the Loss and Accuracy at each of these implementations, resulting in the performance per neuron shown in Figures 1a and 1b. We see that a higher number of neurons leads to higher average accuracy on the validation set, as well as higher average loss (with some fluctuations along the way). This is due to the fact that the increased number of neurons lead to the model being able to learn more complex patterns, which is very important for image recognition, where the features are complex spatial structures and outlines, rather than straightforward numeric values, for example.



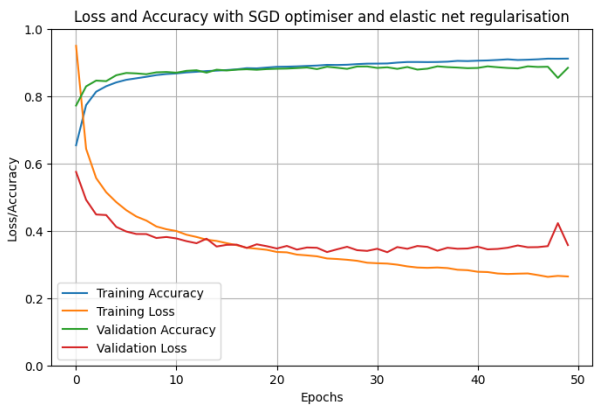
(a) Accuracy per number of neurons

(b) Loss per number of neurons

Figure 1: Accuracy and loss progression per number of neurons for first attempted Multi-Class Perceptron

We proceeded with tinkering with different regularisation procedures, trying L1 regularisation on its own, followed by only L2 and a combination of the two. We also tried different activation functions in the hidden layers, as well as different number of neurons for each of them. We also tried dropout in order to prevent overfitting, as well as normalising the inputs after each layer.

After this, we experimented with changing the optimiser. According to Kingma and Ba (2014), referenced in the Keras documentation, the Adam optimiser is "computationally efficient" and beneficial for problems with "large data/parameters" so we attempt a run with this optimiser. However, the model seems to start overfitting the data earlier than in the runs with the SGD optimiser as can be seen from Figures 2a and 2b, by the fact that the validation loss becomes higher than the training loss much earlier (epoch 2-3) than when using the SGD (epoch 20) so we opt to keep trying further models with the SGD.



(a) "Model 5" metrics

(b) "Model Adam" metrics

Figure 2: Accuracy and loss for Model with SGD VS Adam optimizer

The result was that the best performing model (henceforth called Model 5) had accuracy 0.8840 (in 3 decimals) on the validation set and validation MSE (loss) 0.3574 (in 3 decimals). The kernel is initialised using a random normal initialiser, and the bias is initialised using zeroes.This model uses six hidden layers, each with the ReLU activation function, and different numbers of neurons. Specifically, 64 neurons in the first two dense layers, 128 in the subsequent two dense layers, and finally 32 in the last two dense layers. After each layer, we use normalisation in order to ensure that optimisation is more efficient, as it will not be disturbed by the distributional shift experienced by the weights at each updating by the layers. The output layer, as before, used the softmax activation function to output 10 classes and L1 = 1e-5 and L2 = 1e-4 regularisation. The loss function was sparse categorical cross-entropy, while optimisation was carried out via gradient descent. The results of all statistics for Model 5 are also plotted per epoch in Figure **??**.

Table 1: Model Metrics for Task 1: Multi-Class Perceptron All Models

Model	**Train Acc.**	**Train Loss**	**Val. Acc.**	**Val. Loss**
Two hidden layers and SGD optimiser ("Model 1")	0.9037	0.2703	0.8835	0.3206
Three hidden layers and SGD optimiser ("Model 2")	0.9098	0.2478	0.8875	0.3248
Six hidden layers and SGD optimiser ("Model 3")	0.9136	0.2351	0.8787	0.3588
As above and Elastic Net Regularisation ("Model 4")	0.9338	0.2779	0.8778	0.4867
As above and Dropout ("Model 5")	0.9097	0.2675	0.8883	0.3406
As above but Adam instead of SGD ("Model Adam")	0.9305	0.1837	0.8840	0.3574

However, as we see in the next section, the perceptron algorithm is not very suitable for such a task. Hence , while we applied it to the CIFAR-10 dataset we do not discuss the results in detail.

2.3 Convolutional Neural Network

Next, we applied a Convolutional Neural Network (CNN) to the Fashion MNIST dataset to compare its performance to that of the Multiclass Perceptron. We ran many different experiments, but for brevity, instead of reporting the details of each, we report what we found out was the impact of each hyperparameter on the model. Firstly, we started with a lower number of hidden layers (two) and steadily increased (conservatively, to six due to computational/time constraints), and thus watched the performance improve as the number increased. This is due to the fact that the Convolutional layers learn spatial patterns in the images, therefore, more layers allow for different kinds of patterns to be learned.

Next, we also tried different numbers of filters per layer, which changes the kinds of patterns which each layer is able to learn. Larger kernel sizes "dispersed" the effects that the filter was able to pick up, while smaller kernels focused more on the local differences. Finally, to reduce overfitting we also experimented with normalisation as before, as well as dropouts. Dropout is a technique whereby a certain percentage of the input pixels are left out of training at each layer in order to prevent overfitting by making sure that the patterns the network observes are not overly-dependent on certain pixels.

Finally, to reduce complexity and therefore speed up computation as well as improving the generalisability of the rules produced by our model, we also implemented pooling layers. Pooling layers are implemented as the latter steps of the training procedure and condense the learned patterns into smaller sets of rules. Naturally, this speeds up computation, but also allows for these more "condensed" patterns to be more generalisable.

All these considerations resulted in our best performing CNN model with six hidden layers, each followed by batch normalisation and activated using ReLU. After every two layers we also applied a Dropout rate of 0.3 (corresponding to 30% of the entries). The output layer used a 10-class softmax activation to produce our 10-class predictions. Finally, to compile the model, we used a Gradient Descent Optimiser, and the sparse categorical cross-entropy loss.

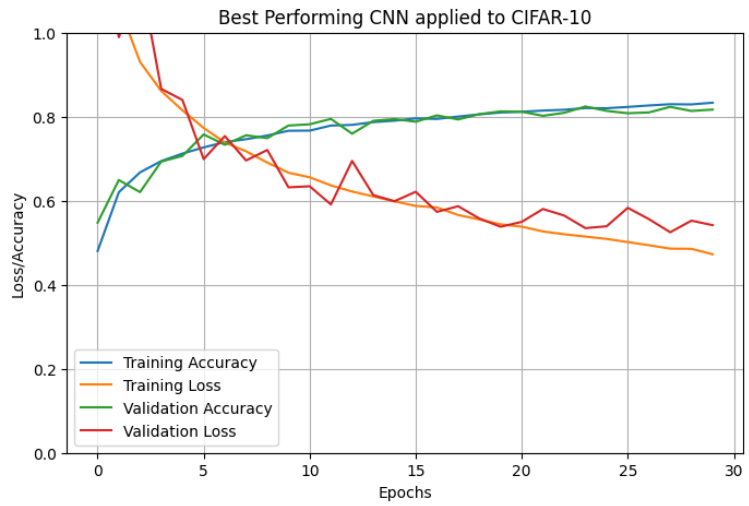


Figure 4: Best performing CNN Model metrics on CIFAR-10

After training for 30 epochs, the lowest validation loss achieved was 0.189 (in 3 decimals) and the highest accuracy was 0.934 (in 3 decimals). The results for each epoch are plotted in Figure 3

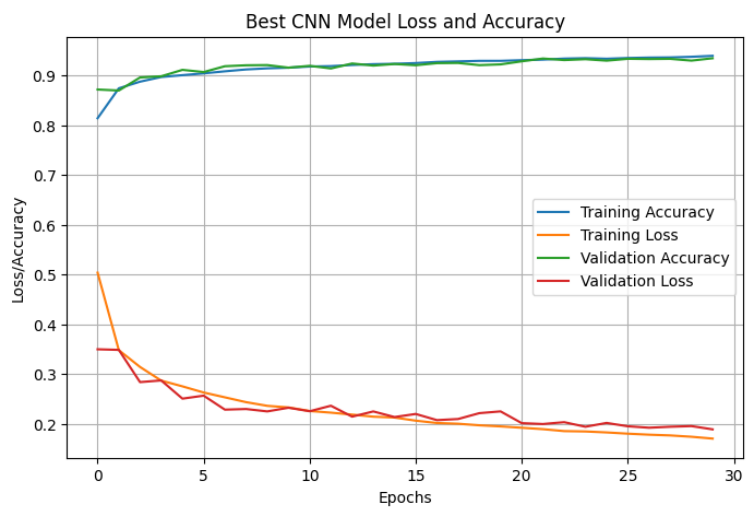


Figure 3: Best performing CNN Model metrics on Fashion MNIST

2.4 CIFAR-10 Dataset

To round off Task 1, we apply our three best performing models to the CIFAR-10 dataset. It contains 60000 images, also of 10 classes, but with a lot more complexity than the Fashion MNIST. The first key difference are the colour channels, as these images are not in black-and-white, allowing for more complex patterns to emerge. As such, the colour intensity for each pixel is no longer represented by a single number on the 0-255 scale but by three different numbers representing the intensity of each of the blue, green or red channels at that pixel. Secondly, the number of pixels is 32 x 32 – slightly more than the previous dataset. Finally, the objects depicted in the images are different: namely animals and vehicles, therefore the patterns needed to categorise these are different than the patterns needed to categorise articles of clothing.

Again for brevity, we discuss in detail the performance of our best model on this dataset. This CNN is the same as described above, but on the CIFAR-10 dataset performs much differently. Its performance across 30 training epochs is presented in Figure 4. Firstly, the highest validation accuracy reached is only 0.824 (in 3 decimals), and the lowest validation MSE is 0.535 (in 3 decimals). The accuracy is almost 10% lower than it is on the simpler dataset, while the validation loss is almost 40% higher.

Table 2: Overview of Training and Validation Loss and Accuracies for Models on CIFAR-10

Model	**Train Acc.**	**Train Loss**	**Val. Acc.**	**Val. Loss**	**Test Acc.**	**Test Loss**
Sequential (6 layers)	0.5666	1.2093	0.4718	1.5275	0.4704	1.5134
Sequential (6 layers with normalisation and dropout)	0.4481	1.5439	0.4436	1.5456	0.4465	1.5239
Convolutional	0.8005	0.5653	0.8020	0.5662	0.8041	0.5732

2.5 Discussion: Why the Difference in Performance between different Datasets?

As we can see in Figure 2, the best-performing model in one dataset does not translate to great performance in another dataset. When comparing these two datasets in particular, the difference could be attributed broadly to three key reasons.

2.5.1 Object Types and Themes

As we discuss earlier, CNNs learn patterns in a 2-dimensional grid by sliding across it and, in each layer, trying to identify a new way in which the image varies from area to area. Due to this, the kind of layers and kernel that will be useful in detecting

the outlines of clothes in the Fashion-MNIST dataset are not the same as those which will be useful when outlining animals and vehicles. The former have perhaps sharper contrasts owing to the fact that they are product pictures photographed against a blank background, whereas the latter are more challenging to separate from the natural background. Since the patterns which CNNs learn are not clearly understandable to humans, it is impossible to predict whether a layer which greatly improved performance in one dataset can replicate that result in another dataset, as we do not know what kind of pattern that layer is responsible for detecting.

2.5.2 Colour Channels

Secondly, the CIFAR-10 dataset contains colour, whereas the Fashion-MNIST dataset does not. This in turn means that the patterns learned in the greyscale setting will not necessarily translate well to the colour setting, since patterns are now dispersed across three channels of pixel intensity than one. Moreover, this increases the complexity of the dataset, as there are more dimensions across which the model needs to find and generalise patterns.

2.6 Category Differences

The Fashion-MNIST dataset is more homogeneous in terms of the products it depicts: the difference between two images of coats is arguably lower than the difference between two images of airplanes. There are only so many ways in which a clothing item can be modified while still remaining functionally the same item, whereas a bird could be any of thousands of species, while still needing to be classified in the "bird" category.

2.7 Discussion: Why the Difference in Performance on the same Dataset?

The above tasks also show that CNNs perform better at image recognition tasks than Perceptrons. As can be seen in Figure 2, our best performing CNN has much higher test accuracy than our best performing perceptron. The reasons for this are chiefly: spatial awareness and variance/variability. We discuss each of these properties in this section.

2.7.1 Spatial Awareness

Images are representations of objects or subjects in space. This means that within an image we are chiefly interested in detecting shapes and colour variations, which signify the outlines of objects. As such, neighbouring pixels are not statistically independent of each other, and detecting a point in the image which contains adjacent pixels with very different channel values should indicate to the model that this part of the image is important as it could delineate the outline of an object. While perceptrons are able to conceptualise that certain values of certain pixels increase or decrease the probability of an object belonging to a certain category, whether the pixels are spatially related to each other is not something that the model is able to pick up on.

On the other hand, convolutional neural networks learn from the two-dimensional data, and do not first have to flatten them into an array. They use kernels which "swipe" across the image, meaning that pixels in close proximity to each other contribute to the filters that the model creates and learns from.

2.7.2 Variance

Convolutional neural networks are powerful in the context of image recognition because of the fact that certain layers have the ability to make them equivariant and invariant to transformations. An image rotated 20 degrees clockwise should not change its classification results when fitted using a convolutional neural network. This is due to the fact that the initial, 2-d convolutional layers use filters to slide across the image and learn where edges, shapes and colour/channel intensity differentials are present. This achieves equivariance: when an image is transformed in some way, the filter is able to detect that what it recognised as an edge has now simply been rotated 20 degrees. Then, pooling layers can achieve invariance to transformations: whether the pattern has shifted, been reflected or is present many times in the same image doesn't matter because it is no longer dependent on the individual pixels it was conceived from, but rather an abstraction of them. Thus, now that pattern could be recognised anywhere within the image.

This starkly contrasts with the perceptron also, which definitely depends on individual pixels and their strict placement in the same position in the flattened array to learn patterns.

3 Task 2: Build a Tell-the-Time Network

After discussing all the pros of CNNs in image-recognition related tasks, we set out to build a network which can read the time from greyscale pictures of analog clocks. The key challenge in this is that the images contain noise: with reflections in the face of the clock, and the clock not always facing in the same direction. We follow different approaches and compare them in the proceeding section.

3.1 Dataset

Our dataset consists of 18000 images, and we are given the inputs it in two versions: a smaller one made up of 75x75 pixel representations, and a larger made up of 150x150 pixels. The labels are two-dimensional, with the first entry for each image representing the hour, and the second representing the minute. We begin by training our models on the smaller-complexity dataset and apply the best ones to the larger for this report.

3.2 Classification

Firstly, we could treat this task as a Multi-Class Classification problem, as above. Specifically, we can segment the clock into intervals of time and then task our network with predicting which time-interval-class the clock seems to be pointing. In order to do this, we combine the hour and minute labels and split them into different-sized categories (for example, our first attempt

split them into 24 half-hour intervals, and subsequently into 48 15-minute intervals, and then into 72 10-minute intervals. While it seems that we could continue this split until we are able to predict any time, this is not the case, as then each class would have too few observations – and essentially nothing to train on. The only remedy for this would be implementation of transfer learning. That is, to train a model on 24 hour intervals, freeze some of this models layers and let other adapt to the data now split into 48 and subsequently 72 groups. Both the results for the transfer and non transfer learning models are discussed.

3.2.1 Model building

Base model Our best classification model without transfer learning was trained on 24 groups. This model contained 3 convolutional layers of an increasing set of filters: 16, 64 and 128 respectively, inspired by the book *Understanding Deep Learning* (Prince, 2023). Higher filter sizes were tried but this resulted only in lower accuracy and MAE. Each convolutional layer used a stride of one together with a kernel size of 3x3. All these layers were followed by a max pooling layer to reduce dimensionality and make the model more invariant to transformations.

Initially, the model exhibited training instability: the performance of the model depended greatly on the random initialization of the model, more often than not the model would fail to pick up on any features of the dataset. To this end, a batch normalization layer was added after the last convolutional layer. This addition stabilized training and made the model consistently converge on subsequent runs. The batch normalization layer could either be added before or after the activation layer following the convolutional layer. Both scenarios were tested but in this particular case placement seemed to have little effect on the model's performance, thus we added batch normalization in between the convolutional layer and the activation layer. Furthermore a dropout layer of 0.5 was added after flattening to combat overfitting and bring test and train accuracy more on par with each other. Finally, the model concluded with two dense layers of 128 units followed by a softmax output layer of 24 units. More hidden dense layers were tried out but the small increase in accuracy did not seem to justify the larger increase in complexity.

Another issue that plagued this model was the fluctuation of validation loss and accuracy introduced by the addition of batch normalization (Figure 5). To remedy this a lower learning rate of 0.005 and higher epoch size was tried out. While this dampened the fluctuations only by a little, it did improve the models performance and thus was kept.

Models were also trained on 48 groups and 72 groups but, as expected, model performance declined with a higher number of groups due to increasing sparsity of the data. This resulted in models that would have a lower accuracy, MAE and overall unstable behaviour.

3.2.2 Transfer learning models

In order to increase accuracy while predicting smaller intervals (and therefore a larger number) of groups, we had the idea to implement transfer learning. Transfer learning uses models which were trained on similar (usually simpler) tasks and trains them on a new dataset, where the model can retain some of the old learned rules, but also create new ones from the unseen data. Our aim was to replicate the better performance that our models displayed on the smaller amount of groups, on the larger group sizes.

Given the model trained on 24 groups performed the best, it was also used as base for the transfer learning approach. Application of transfer learning went in two steps: first the model was trained to recognize 48 groups and finally this new model was trained to recognize 72 groups. As per recommendation of *Understanding to Deep learning* (Prince, 2023) the output layer of the base model was skipped and all other layers were used as building blocks. After an iterative process we concluded that keeping all layers trainable resulted in the best performance. Three hidden dense layers of 265 units were added in the same manner.

3.2.3 Results

The results of the best performing models are shown in the following table.

Table 3: Overview of model accuracy and loss for classification models

Model	**Train Acc.**	**Train Loss**	**Val. Acc.**	**Val. Loss**	**Test Acc.**	**Test Loss**
24 Batch normalization Lowered learn rate	0.9115	0.251	0.853	0.473	0.364	8.158
24 to 48 transfer learning	0.9354	0.1847	0.9086	0.2753	0.8506	0.428
48 to 72 transfer learning	0.9424	0.1749	0.9025	0.2702	0.8950	0.285

Table 4: Overview of "Common Sense" MAE and error time for classification models

Model	**Val. MAE**	**Val. Minutes**	**Test MAE**	**Test Minutes**
24 Batch normalization Lowered learn rate	0.0301	21.640	0.078	56.251
24 to 48 transfer learning	0.0142	10.244	0.019	13.517
48 to 72 transfer learning	0.0098	7.079	0.009	6.768

As previously stated, one issue that arose with the base 24 model was the fluctuation of the validation error and validation loss (Figure 5). This is most likely caused by the implementation of batch normalization, as models without these layers do not have this issue. Our interpretation of the reason for this has to do with how batch normalization is carried out in Keras during training versus during validation. According to Ioffe et al., as referenced in the Keras documentation for batch normalization, while training the model, the batch normalization layers takes mini batches of activations and normalizes the activations by calculating the mean and variance of that batch. However, during inference, the batch layer "normalizes by using a moving average and and variance over all previously calculated batch statistics" (Ioffe and C., 2015). This discrepancy in train and validation could explain why the fluctuations appear. If the individual batches are noisy, too small, or in any other way do not represent the population statistics it would result in individual batch statistics differing from each other. This would then create unstable moving mean average and variance, leading to fluctuations in validation loss and accuracy. This inherently makes these models less reliable, as its prediction on new data will vary a lot, seen in how big the difference is between the validation MAE and test MAE. An issue that will only be

magnified when splitting the dataset into more groups as this would create even less training data per group, inevitably resulting in worse performance.

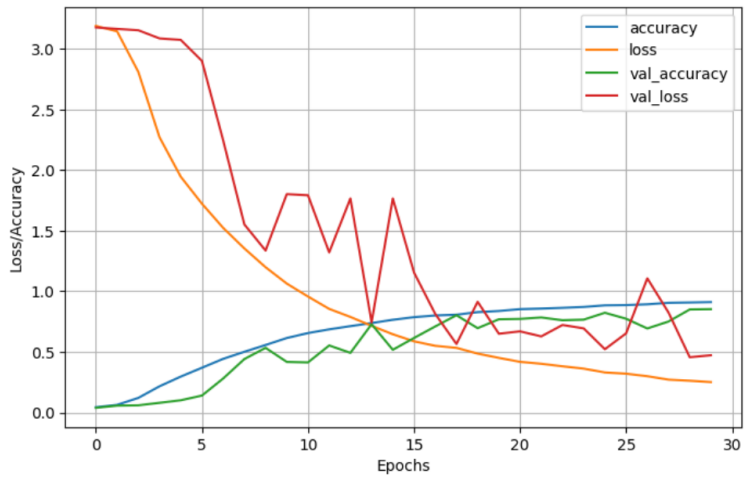
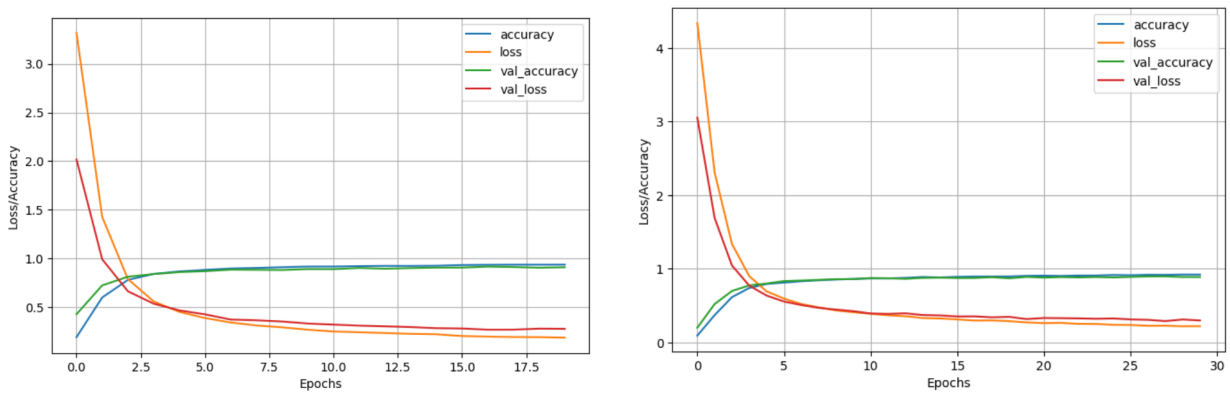


Figure 5: Accuracy and loss of classification model trained on 24 groups

Interestingly, the transfer models do not seem to deal with fluctuation issues even though they have been built upon the previous model. In essence, the batch normalization layer tries to estimate the population mean and variance and normalize each batch using these estimates during inference. In the base model, these statistics have to be estimated from scratch over different, potentially noisy batches, leading to instability. In contrast, the transfer learning model already has a starting point, the batch normalization statistics from the base model. Even though this starting point is shaky, it is still better than no starting point, potentially resulting in the transfer model being able to more precisely estimate the population statistics. Regardless, it is evident that the transfer models outperform the base model, with the 48 to 72 transfer learning model achieving an 6.768 average minute error on the test set.



(a) 24 to 48 groups

(b) 48 to 72 groups

Figure 6: Accuracy and loss of classification transfer models

Our models were also tested on the 150x150 images, but on this data the models performed subpar. The base model trained on 24 groups, the transfer model from 24 to 48 groups and the transfer model from 48 to 72 groups having an average minute error on the validation set of 34, 20 and 14 minutes respectively. This is to be expected as the image itself now contains more data with more sparse areas. The current architecture is not built to capitalize on this. For example, the kernel size of the convolutional layer could be increased if the image size is also increased.

3.3 Regression

Now, we consider this as a regression problem. In order to do this, we convert the outcome sets into float numbers, where the whole part represents the hours in a 12-hour clock (0-11) and the decimal part represents the normalised minutes on a 0-1 scale. After this process, for instance, the time 6:30 becomes as 6.5, and the time 23:45 becomes 11.75. Once the transformation is applied, we can move onto model building. For brevity, we will only briefly explain the model building process, and we will describe the best-achieving model out of all the attempts.

3.3.1 Model building

During model building, we mainly look at two specific activation functions, that employ very different logics. Firstly, is the sigmoid function, which is also involved in the classic logistic regression, and the second is the linear activation function. Both have advantages and disadvantages. The former caps the values at 1, which is good, since the hour-minute values have a limit of 12, if not normlised, and of 1 when normalised; however, sigmoid also takes the vaues of 0 and 1 only asymptotically. This means that not all values between 0 and 1 are assigned the same probability, which is incorrect in the case of a clock. The latter fares better in terms of distribution, as in a linear scenario all values take similar probabilities; however, a linear function does not have a natural maximum value, which means we could have values way over 12, if the model is not well-specified.

This being said, the two best-performing implemented models have the same structure. They are composed of six 2-dimensional convolutional layers and four dense layers. The choice of 2-dimensional convolutional layers stemmed from the nature of the data;

this sort of layer is more likely to capture the patterns between and among the pixels of a 2-dimentional picture, unlike a dense layer, which flattens the input and employs all pixels indiscriminately to formulate the input for the next layer. Following the convolutional layers, we added four dense layers. We made this choice with the issue of generalisation in mind. By adding dense layers at the end, we allow the convolutional layers to only take on the task of feature extraction, as the dense layers at the end act as a 'funnel' that returns a smooth regression-like result.

While ten layers compose a quite deep network, which might be prone to overfitting, we added max pooling after each couple of convolutional layers and dropout after each dense one, that would contribute to mitigating the overfitting issue. Moreover, we added L2 regularisation with a λ of 0.0001, to penlise less relevant weights and improve generalisation to new datasets. Additionally, upon noticing a problem of vanishing gradients, we implemented batch normalisation after each convolutional layer and layer normalisation after each dense one, together with implementing reduction of the learning rate at circular MAE plateau.

3.3.2 Loss and accuracy

As for our loss and accuracy choices, standard classification accuracy would not work in our case. For this model, we chose the Huber loss function as a measure of loss. This was chosen due to its hybrid nature; in fact, the Huber loss function is linear when the residual is lower than the threshold δ, and quadratic otherwise. This means that it both has less sensitivity to outliers, and penalizes small errors within the data points.

As a measure of accuracy, we employ a custom Mean Absolute Error (MAE). This measure works as a "circular" MAE, by keeping into consideration that time is circular, thus calculating the appropriate distance, instead of the absolute one. For instance, having 11:55 as a true value, and 0:03 as a predicted value, traditional MAE would give an error of 11 hours and 52 minutes, while our circular MAE would return 8 minutes. The logic of MAE for our custom metric was chosen due to its interpretability. Compared to the Mean Squared Error (MSE), it is easier to extract a measure of inaccuracy that relates to the data. If the non-normalised MAE is 0.5, we know instantly that, on average, our model misses the true time by 30 minutes.

3.3.3 Results

In table 6, we can see that for the sigmoid model, we find a normalised circular MAE on the train set of 0.0426 and on the validation set of 0.0307, which amounts to ≈22 minutes. For the linear model, we have a train circular MAE of 0.4286, and a validation circular MAE of 0.4549, which is equivalent to ≈27 minutes.

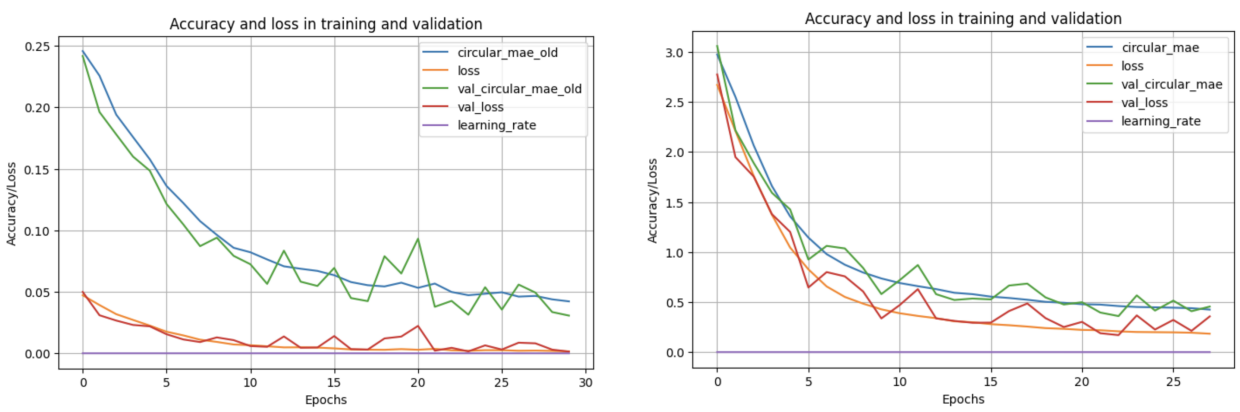
Upon testing the models against the test set, we find a circular MAE of 0.0305 for the sigmoid model and 0.4719 for the linear model. This means that the models, on average, predicts with a respective difference of ≈22 and ≈28 minutes from the true value.

Table 5: Overview of model circular MAE and loss for regression models

Model	Train MAE	Val. MAE	Test MAE	Train Loss	Val. Loss	Test Loss
Sigmoid	0.0426	0.0307	0.0305	0.0017	0.0015	0.0016
Linear	0.4286	0.4549	0.4719	0.1858	0.3566	0.3637

3.3.4 Variance

The two models perform quite well in terms of variance among sets. As can be seen in Figure 7a, the accuracies and losses of the sigmoid model do not have systematically wide gaps. The only issue we can identify from the graph, is the slightly unstable gradients, which show in the peaks and lows towards the end of the curve. For the linear model, the situation is better. We can see that the validation accuracy and loss remain close along the curve, with only small divergences. This indicates that the model has very little variance, and thus not overfitting.



(a) Sigmoid loss function

(b) Linear loss function

Figure 7: Accuracy and loss for best-performing Regression Models

3.4 Multi-Head Approach

A multi-head approach is useful for when the same dataset is required to make different kinds of predictions. For example, in Chapter 10 of Hands-on Machine Learning, Geron (2022) gives examples of when some inputs need to be fed through the deep parts of the network, whereas others need only pass through shallow layers (p. 308-310).

For our task, all features are pixels of images, meaning that such a split into a shallow and deep part of the network for inputs is not what is required. Instead, the two-head approach is described as such due to the fact that it produces two different outputs. Since we are trying to predict time in hours and minutes, our two outputs are predicted hour and predicted minute. This

multi-output approach allows us to specify a different activation function for each head, thus defining a different learning task for each.

The hours are predicted via classification into one of 12 groups, for which we use keras' to_categorical function in order to turn the hour labels into a categorical variable (one-hot-encoding). The minutes are predicted using regression.

3.4.1 Model Building

As mentioned above, our task requires that all inputs pass through the same set of Convolutional layers regardless of whether the output we are seeking are hours or minutes.

At first we begin with four convolutional layers, all followed by batch normalisation and 2-dimensional max pooling layers, based on the lessons learned implementing the previous two types of models. After these layers, we flatten the output. The two-head approach becomes evident at this point, as what follows are two different "branches" each responsible for recognising one part of the time.

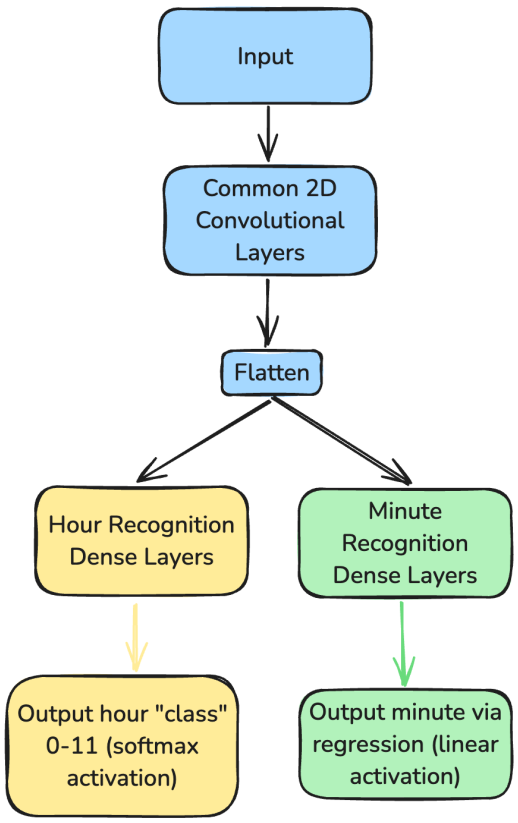


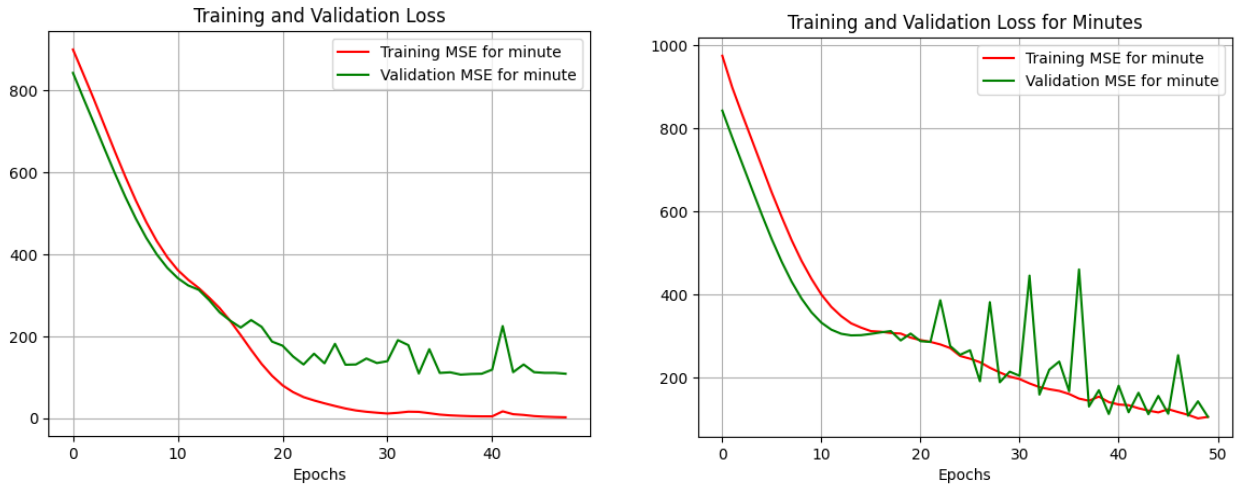
Figure 8: Architecture of the Two-Head Model

For the hour recognition layers we began with two dense layers, followed by layer normalisation and finally ended with a dense layer with 12 probability outputs – each the probability of the hour being 0 - 11 using softmax activation. For the minute recognition layers we also stacked two dense layers, each followed by layer normalisation but of course, had a different output layer, with one possible output: the predicted minute, using a linear activation. Here it should be noted that since our best performing regression utilised the sigmoid activation and standardised minutes, when attempting this during the two-head, it yielded less accurate results, which is why we choose not to standardise the minutes. This phenomenon is likely due to the fact that since the two-head model needs to categorise into 12 hour categories (hence we cannot standardise the hours), standardising the minutes makes the scale of the two losses too different, and thus complicates minimising it for both sets of units.

For two-head models, the loss for optimisation can be different for each output. As such, we optimise the hour (classification) task using Categorical Cross-Entropy loss and the regression task using the Mean Squared Error. Of course, what we are really interested in is maximising the common-sense accuracy for predicting the actual time, therefore for each model we also computed the "common sense" error as in the Classification and Regression tasks above.

We followed an iterative process to select the rest of the model parameters. Firstly, we noticed early on that the model struggled more with predicting the hour accurately than with the minute. Thus, we played around with different weightings of the loss when compiling it so that the model would prioritise minimising the cross-entropy loss for the hour-classification task over reducing the MSE for predicting the minutes (p. 312, Geron, 2022). The final model is weighed 90% for hours and 10% for minutes, as we noticed that before weighting, the training versus validation loss per epoch would diverge early on for minutes as seen in Figures 9a and 10b.

In the weighed model (on the right), the validation loss keeps up with the training loss (barring the fluctuations due to instability), while in the unweighed one (on the left), the training loss keeps reducing while the validation remains stable – signalling overfitting.



(a) Unweighed two-head model (Model 5)

(b) Weighed two-head model

Figure 9: Loss per epoch progression for minutes for an Unweighed versus a Weighed Two-Head Model

Secondly, we noticed that applying normalisation between the dense layers in the hour-recognition branch caused the validation loss to "jump" between epochs, signifying gradient instability. We removed them to rectify this but noticed that this caused our model to overfit, as the reduction in training loss was not accompanied by a reduction in the loss on the validation set. As such, we applied L2 kernel regularisation of 0.001 to the dense layers both in the hour and minute branches to prevent this, resulting in the training and validation loss not diverging from each other as significantly as before.

While we also experimented with using different optimizers, the results were not as promising as with the Adam optimizer, which we also knew from the previous tasks to give quite good results. However, we did reduce its learning rate to try to reduce the amount of "jumping" in the validation loss between epochs. However, some of the models with lower learning rates did not have as much success in terms of the common sense error on the test set than the ones with more unstable gradients.

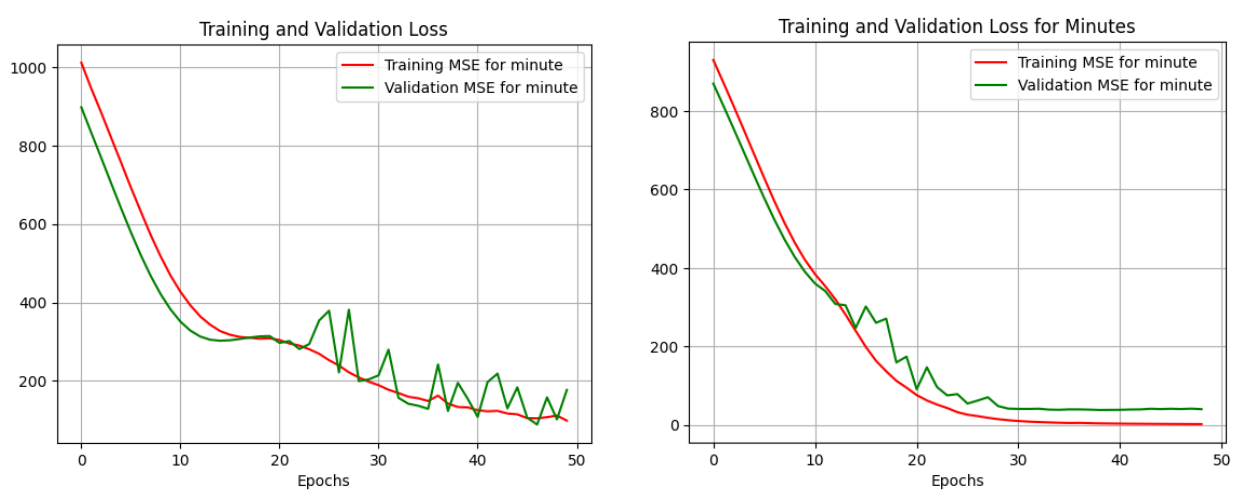
Other design choices we experimented with were early-stopping to prevent overfitting, with a patience of 10 - 20 epochs, since we are aware of the sharpness of the gradients for this problem and did not want the model to converge to a local minimum, while missing a potential global minimum down the line. We also further attempted to reduce overfitting by adding dropout layers.

3.4.2 Residual Networks

After trying many different designs and being disappointed with the seeming instability of the gradients, we decided to implement some residual network architecture following Lectures 7 and 8, in order to try to rectify the problem.

Residual networks prevent shattered gradients by introducing residual connections to our network, which allow some layers to be skipped in case the gradient behaves erratically. Thus, the information passing through layers is more "gradual" allowing for the subsequent gradients to be smoother, and overall training to make more conservative steps.

Adding three skip layers into our network appears to have helped the model tremendously, especially in later epochs, as can be seen in the Figure 10a below, where we compare the loss per epoch for the minutes branch of a model without and a model with skip layers.



(a) Model (5) with no Residual Connections

(b) Model (16) with Residual Connections (3 skip layers)

Figure 10: Loss per epoch for minutes progression for a Model without Residual architecture VS with

3.4.3 Results

The results of some of our best performing models for the two-head approach are tabulated below.

"Model 5" uses four convolutional layers, beginning with a kernel size of 5 and reducing to two for the final layer, and beginning with 30 filters and increasing to 300 for the final layer. Each of these were followed by Batch Normalization and Maximum Pooling. It has two dense layers for hour and minute recognition branches, each also followed by Dropout at rate 0.3 and ending in Layer Normalization.

"Model 6" is as above, but also weighs the minute versus the hours branch, giving them weights 0.3 and 0.7 respectively.

"Model 14", achieves a common sense error on the test set of 10.75 minutes. It has five convolutional layers, each followed by Batch Normalization and MaxPooling, and begins with 30 filters, increasing all the way to 360 at the final convolutional layer. However, in this model, before flattening and moving onto the dense layers, we also have three skip layers which use additive merging to combine the outputs of convolutional layers two and three, three and four, and four and five respectively, thus creating shortcut paths that help gradient stability during backpropagation. Finally, it also weighs the two tasks with hours taking 0.9 and minutes 0.1 to prevent overfitting for minutes.

Of course, many more models were tried but we do not describe them for brevity.

On the 150x150 dataset, our best performing model fares worse, with a common sense error in minutes of 29.5511. However, this makes sense as the features that are useful for a more "dense" feature space such as the 75x75 images, are not necessarily the same as would be useful on the sparser 150x150 dataset. For future applications, we might consider tuning the model further on this dataset, similar to how we applied transfer learning.

Table 6: Overview of Losses on the Validation Set for (some of) the Two-Head Models

Model	**Cross-Entropy Loss (Hours)**	**MSE (Minutes)**	**Common Sense Error (in Minutes) on Test Set**
Model 5	1.8009	176.9031	62.7005
Model 6	1.6068	106.7705	55.9977
Model 14	0.7997	27.4521	10.7455

3.5 Discussion

The common sense error of this final two-head model is notably better than the regression models, but only better than the classification models not using transfer learning. This is due to the fact that the two heads are able to "specialise" in their own predictive tasks. Since loss-minimisation for hours is a different task than loss-minimisation for the minutes, meaning that the model does not have to compromise how well it does at predicting the hours in order to improve its predictions for the minutes. While the transfer learning classification model does achieve a slightly lower common sense error (approximately 7 minutes versus 13), we were not able to use the transfer learning technique we used in the classification model, as the hours cannot be split into more categories than 12 without interfering with the minute task.

This is (partly) due to how optimization takes place. The two heads are minimizing two separate functions' gradients. If this was a task predicting both minutes and hours, there could be a step in the gradient space which would improve the prediction for minutes but make the prediction for hours worse. The optimizer then may choose not to "move towards" this direction in order not to compromise the prediction for hours. However, with two heads, the optimizers are able to move in any direction that improves predictions for hours and minutes separately.

4 Conclusion

Task 2 shows the importance of tailoring the architecture of our Neural Networks to the task at hand.

Using transfer learning we were able to acquire the lowest common sense error of all the models, by decreasing the amount of 'guessing' required by our optimizer when training begins. By allowing our model with more categories to begin minimising the loss at a point in the space where it knew the loss was minimal for a simpler task, it could more easily reach a minimum for the more complicated task of classifying more groups.

Additionally, we have successfully implemented measures against overfitting, which would be an otherwise prominent issue, especially with the regression models. A combination of regularisation, dropout, and pooling, allowed us to create models that displayed minimal variance, and that are thus capable of generalisation.

Moreover, using the two-head model, we were able to take advantage of some of the best practices learned both during the regression and classification tasks, and combine them in order to reach a better prediction for the time by predicting the hours as a category and the minutes as a regression task separately. Again, loss minimization was more efficient in this approach as the model did not need to trade-off minute accuracy for hour accuracy, and could optimize for both tasks separately but combine them into a single prediction.

References

Géron, A. (2022). *Hands-on machine learning with scikit-learn, keras, and tensorflow*. O'Reilly Media, Inc.
Ioffe, S., & C., S. (2015). Keras documentation.
Kingma, D. P., & Ba, J. (2014). A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, *1412*(6).
Prince, S. J. (2023). *Understanding deep learning*. MIT press.